Generalisation and Adaptation in Reinforcement Learning: A Comparative Study of Algorithms in the LunarLander Task

Michael Yianni

1 INTRODUCTION

Reinforcement learning (RL) trains agents through interaction with an environment to maximise cumulative reward. However, good training performance does not guarantee robustness when deployment conditions differ from those seen during training. This makes generalisation and transfer learning important practical concerns.

This report investigates these issues using the LunarLander [1] task from Gymnasium [2], a widely used benchmark for reinforcement learning [3], [4] in which an agent must control a spacecraft and land it safely on a designated pad.

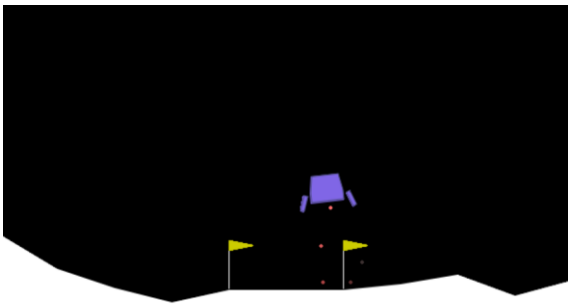

Figure 1: *Screenshot of the LunarLander environment.*

Three reinforcement learning algorithms are examined: Deep Q-Networks (DQN), Proximal Policy Optimisation (PPO), and Advantage Actor-Critic (A2C), as well as an Ensemble Agent that combines their decisions via majority voting.

This report addresses two research questions. RS1 asks:

"How do different reinforcement learning algorithms generalise to modified environments in the LunarLander task?"

To answer this, agents are trained in the standard environment, frozen, and then evaluated in perturbed versions featuring increased gravity, wind disturbance, and sensor noise. RS2 asks:

"How does transfer learning from a standard training environment affect the re-convergence speed and final performance of different reinforcement learning algorithms when deployed in modified LunarLander environments?"

To investigate this, pre-trained agents are fine-tuned in each modified environment and compared against training from scratch and the pre-trained frozen models. Together, these experiments aim to provide a comparative analysis of robustness, generalisation, and adaptability in reinforcement learning.

2 BACKGROUND

2.1 REINFORCEMENT LEARNING & LUNARLANDER

The main components of RL are the agent, the environment, and the goal, with the objective being to determine the relationship between these [5]. Literature widely models this using the Markov Decision Process (MDP). An MDP defines the interaction between an agent and its environments in terms of states, actions, transitions, and rewards [6]. In LunarLander [1], the state is represented by an 8-dimensional vector describing the lander's position, velocity, angle, angular velocity, and leg contact statuses. The action space is discrete, with four possible actions: do nothing, left thruster, right thruster, and main thruster. The reward function is designed to guide learning, rewarding moving closer to the landing pad, reducing speed, maintaining a stable orientation, while fuel usage incurs penalties. Successful landings yield a high final reward, whereas crashing is strongly penalised.

2.2 ALGORITHMS

The proposed environmental modifications create distribution shift: when "there is a difference between the training and test distributions" [7]). A good algorithm set should span different reinforcement learning paradigms and stability properties. Thus, the following RL algorithms were chosen:

- Deep Q-Networks (DQN)
- Advantage Actor-Critic (A2C)
- Proximal Policy Optimisation (PPO)

DQN [8] is a value-based, off-policy method [9]. It uses a neural network to estimate the Q-function, which "evaluate(s) the expected rewards of taking a particular action in a given state" [10]. The algorithm stores a replay buffer that stores past transitions. Random minibatches are sampled from this buffer periodically, using each sample's reward to perform gradient descent on the total loss to update the target network.

PPO [11] is an on-policy, actor-critic, hybrid method that combines elements from both value-based and policy-based approaches [12]. It learns both a policy (the actor), that maps the state to a probability distribution over the action space, and a value function (the critic), which evaluates the expected future reward of the current policy. The critic's state-value estimates are used to compute advantage estimates of minibatch samples in order to determine the loss of the policy and update it using gradient descent. PPO is designed to improve training stability by constraining how much the policy can change in a single update via a clipped surrogate objective function, reducing the risk of destructive updates.

A2C [13] is also an on-policy, actor-critic, hybrid method [14], but it is simpler than PPO because it does not use clipped policy updates, and it does not use minibatch sampling: it uses the whole batch to update the policy once, and discards the data.

These algorithms are often compared in recent studies [3], [4], [15]. Their differences are directly relevant to the proposed research questions, because each algorithm's update rule, data usage, and

stability mechanisms determine (a) how much its learned behaviour relies on fine-tuned dynamics from the training environment, and (b) how safely and efficiently it can adapt when re-trained.

Additionally, this project introduces an ensemble agent. The ensemble combines the action predictions of DQN, PPO, and A2C using majority voting. Different algorithms may make different errors, and combining them may reduce the impact of individual failures, potentially improving robustness in modified environments. Ensemble Reinforcement Learning remains a relevant topic in the current RL landscape [16].

2.3 Transfer Learning

Transfer learning is "a machine learning technique in which knowledge gained through one task or dataset is used to improve model performance on another related task or different dataset" [17]. The aim of transfer learning is to determine whether prior learning can be reused to accelerate adaptation in a new environment through fine-tuning. In this study, the source task is the standard LunarLander environment, while the target tasks are the modified environments. To assess whether the prior knowledge is genuinely useful, fine-tuned agents are compared against scratch controls: agents trained from scratch in the target environment. If fine-tuning leads to faster recovery or better final performance, this indicates positive transfer and vice-versa for negative transfer [18].

3 Related Work

LunarLander and other classic control tasks are widely used in RL because they are reproducible, interpretable, and sufficiently complex to reveal differences in algorithm behaviour. Prior studies have used such benchmarks to compare methods including DQN, PPO, and A2C in terms of reward, convergence, and stability [3], [4]. However, this work often focuses mainly on performance in the training environment.

Recent reinforcement learning literature has increasingly argued that strong performance in the training distribution does not necessarily imply robustness under distribution shift [19], and that many existing RL systems are "not designed with generalisation in mind" [20].

Transfer learning addresses a related question: whether knowledge learned in a source task can improve learning in a related target task. One study notes that "acquiring sufficient interaction samples can be prohibitive", and that this challenge has "motivated various efforts to improve the current RL procedure", resulting in transfer learning becoming "a crucial topic in RL" [21].

This project builds on these themes by comparing baseline performance, frozen-policy generalisation, and fine-tuning in modified LunarLander environments, while also including scratch-trained controls and an ensemble agent. This allows the potential source-task performance, robustness, and adaptability to be distinguished within one controlled benchmark.

4 Methodology

4.1 Implementation

All agents were implemented using the Stable-Baselines3 Python library [22], which provided standardised implementations of DQN, PPO, and A2C. This ensured consistency across the algorithms. Custom wrappers were built for these to provide a common interface for training, saving, loading, and evaluation. Modified environments were built from Gymnasium's LunarLander, and NumPy [23] and MatPlotLib [24] were used for data collection and visualisation.

Instead of requiring training, the ensemble agent uses three existing models (one of each algorithm). Each model is loaded and stored, and their predictions for each action are aggregated to determine the majority vote with PPO's vote acting as the tiebreaker.

4.2 Environment Design

Three modified environments were used to test robustness beyond the training distribution: increased gravity (-13.0 instead of -10.0), wind disturbance (*wind_power = 15.0*, *turbulence_power = 1.5*), and sensor noise (standard deviation of 0.075). Unlike the other perturbations that are supported by LunarLander's built-in parameters, sensor noise required a custom implementation, applying Gaussian noise to the output state at every timestep. These environmental parameters were experimented with to balance the extremity of each environment without negatively impacting the performance of each algorithm so much that all analytical resolution is lost.

These environments were intentionally chosen to represent different forms of distribution shift. Increased gravity and wind create dynamics shifts [25], whereas sensor noise creates an observation shift [26]. This distinction is important because different algorithms may vary in response to different distribution shifts.

4.3 RS1: Generalisation

RS1 investigates generalisation. Each algorithm was first trained on the standard LunarLander task whilst logging the reward curves (reward vs the number of episodes) to gauge learning behaviours. The model parameters were then frozen and the resulting policy was evaluated for 100 episodes in four settings: the original standard environment, and the three modified environments described in *Section 4.2*. By keeping the policies fixed during this evaluation, any reduction in performance can be confirmed as a failure of generalisation as opposed to a consequence of insufficient retraining.

4.4 RS2: Transfer Learning

RS2 examined transfer learning and adaptation. For each modified environment, the frozen models pretrained on the standard LunarLander task (from *RS1*) were fine-tuned in the new environment. To provide a control, a separate agent of each algorithm was also trained from scratch in each modified environment for the same duration as fine-tuning.

Fine tuning and scratch training were both run for 40% of the standard training budget to measure the value of a pretrained starting point in terms of efficiency. Final evaluation of these was then carried out over 100 episodes, and additionally compared with their frozen counterparts. The reward curves from the fine-tuning and scratch training were also logged and used to compare their recovery trajectories to determine how quickly agents improved and whether they approached or exceeded their frozen policy performance.

4.5 Metrics

Several metrics were used in order to assess performance, consistency, and adaptation quality. For performance and consistency:

- Average episode reward
- Success rate (%)
- Median reward
- Standard deviation of rewards

A successful run is defined by the LunarLander environment as an episode with 200 reward or above [1].

For learning and adaptation quality and efficiency:

- Learning curves (reward vs number of episodes)
- Learning efficiency (number of episodes required for convergence)

Together, these metrics allowed comparison not only of which algorithm performed best, but also of how reliably they performed and how effectively they adapted under environmental change.

5 EXPERIMENTAL SETUP

5.1 TRAINING BUDGETS

The three baseline agents were not all trained for an identical number of timesteps, because early test runs showed that their learning efficiencies differed substantially. DQN was trained for 500,000 timesteps, which was sufficient for convergence. PPO was trained for 1,000,000 timesteps as its learning curve continued to improve beyond 500,000 timesteps – see *Figure 2*. The A2C agent's learning behaviour was more complex. Initially with a training budget of 500,000 and using standard hyperparameters, A2C suffered from policy collapse.

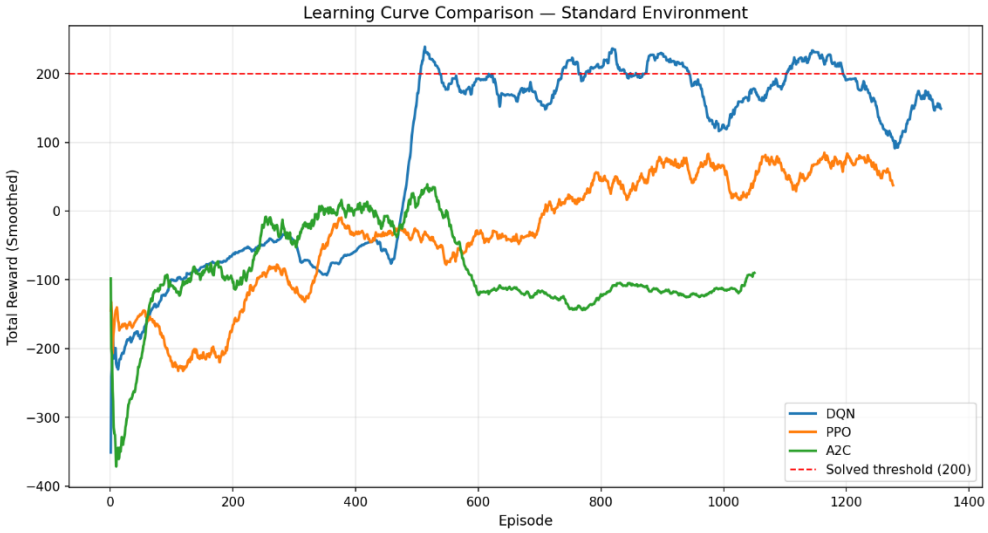

Figure 2: *Initial Learning Curve of the RL Agents after 500,000 timesteps in the experimentation phase (not final).*

This behaviour is consistent with the instability of vanilla policy gradient methods under large updates – a limitation that motivated the development of PPO [11]. Consequently, A2C underwent further hyperparameter tuning, detailed in *Section 5.3*. Review of existing experimentation with A2C in the LunarLander environment [15] also displayed this policy collapse, and that this algorithm required around 5 million timesteps to converge.

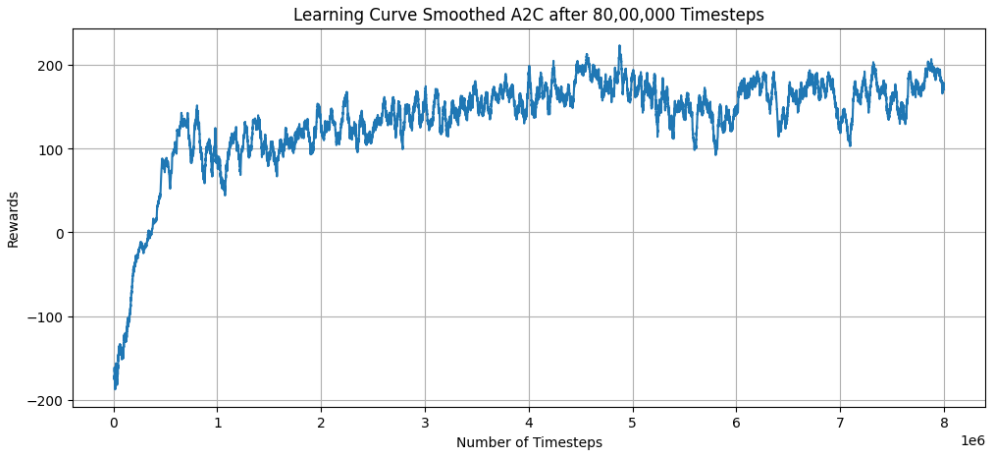

Figure 3: *Learning Curve for A2C after 8,000,000 timesteps. Source: reproduced from* [15].

Thus, an experiment was conducted by training the A2C agent for 5 million timesteps to evaluate whether its performance would significantly improve.

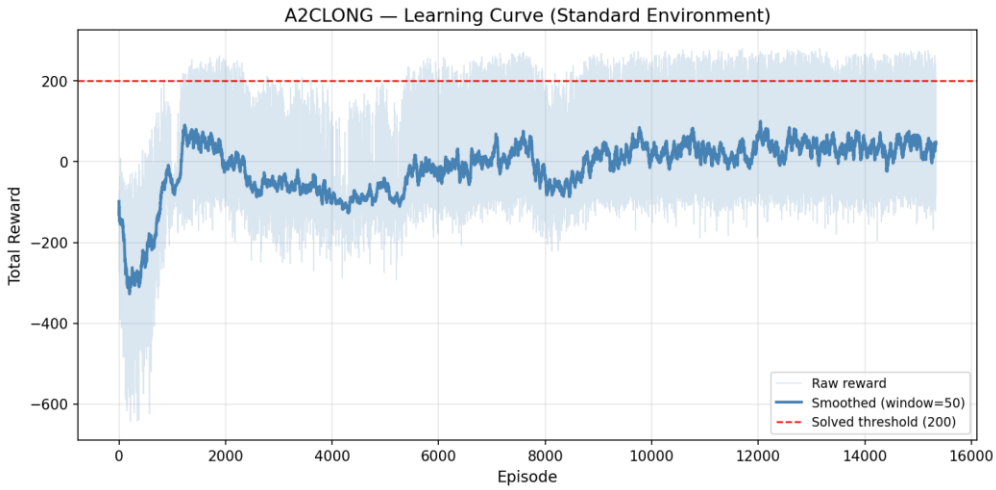

Figure 4: *Initial Learning Curve of the RL Agents in the experimentation phase (not final).*

The results showed that the total reward converged around 50, remained unstable, and did not justify the extra training time above the other models. It was decided that a training budget of 500,000 would be used, stopping training at A2C's peak performance before policy collapse such that the most competent model possible would be used for evaluation.

5.2 HYPERPARAMETERS

Hyperparameters were chosen using a combination of Stable-Baselines3 defaults, values commonly used in the reinforcement learning literature, and tailoring to the LunarLander task. A full grid search was outside the scope of this project, so the aim was instead to select stable and defensible settings that allowed a meaningful comparison between algorithms.

Table 1, 2, and 3 - Hyperparameter values:

DQN

Parameter	Value
policy	MlpPolicy
learning_rate	1e-3
buffer_size	50,000
learning_starts	1,000
batch_size	64
gamma	0.99
train_freq	4
target_update_interval	250

PPO

Parameter	Value
policy	MlpPolicy
learning_rate	3e-4
n_steps	1024
batch_size	64
n_epochs	10
gamma	0.99
gae_lambda	0.95
clip_range	0.2

A2C

Parameter	Value
policy	MlpPolicy
learning_rate	3e-4
n_steps	64
gamma	0.99
gae_lambda	0.95
ent_coef	0.01
vf_coef	0.5
max_grad_norm	0.5

Mainly, the hyperparameters were selected based on the SB3 implementation defaults, which are said in the documentation [2] to be explicitly derived from relevant academic papers in the field [11], [13], [27]. However, pragmatic adjustments to DQN's and A2C's hyperparameters were tailored to LunarLander. DQN's default values are configured for the Atari environment, which has a much higher observation space dimensionality (84 × 84 × 4 image [8]) than the LunarLander environment. Thus, the default buffer size was decreased from 1,000,000 to 50,000, the learning rate was increased from 1e-4 to 1e-3 (as the smaller MLP network is less sensitive to larger updates), the batch size was increased to from 32 to 64 (as there is no longer a memory constraint, and a larger size provides lower variance), and the target update interval was decreased from 10,000 to 250 (to increase update frequency and keep the training budget low). Using these values, DQN displayed successful results in the standard environment, so further tuning was not required.

5.3 A2C TUNING

A2C required additional because initial runs were unstable (see *Figure 2*). Three main changes were introduced. First, *n_steps* was increased from the default 5 to 64, reducing the likelihood of poor performing batches of data inducing bad updates. Second, *gae_lambda* was reduced to 0.95 from 1, introducing Generalised Advantage Estimation to trade a small amount of bias for lower variance. Third, *learning_rate* was reduced to 3e-4 from 7e-4 to make policy updates less aggressive and reduce the risk of destructive updates. Together, these modifications improved training stability sufficiently for A2C to be included in the final comparison.

6 RESULTS

6.1 BASELINE TRAINING & STANDARD PERFORMANCE

The results from this stage establish the baseline competence of each algorithm and provide the reference point for both *RS1* and *RS2*.

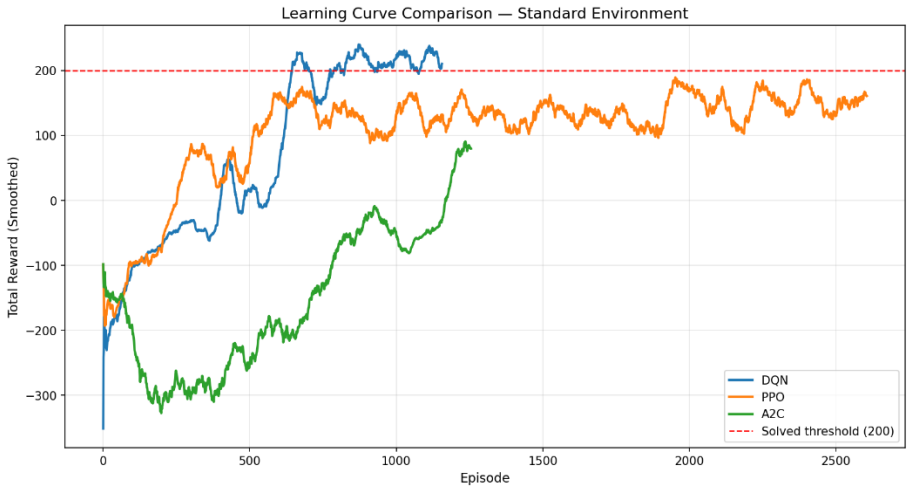

Figure 5: *Learning Curve of DQN, PPO, and A2C in the standard LunarLander environment.*

Figure 5 shows that DQN and PPO improved rapidly during standard-environment training, while A2C appears to learn much less efficiently. Although it appears that A2C is prevented from further improvements from *Figure 4*, its training budget is justified in *Section 5.1*. DQN converged above the solved threshold, PPO approached but remained below it, and A2C showed weaker and less stable learning.

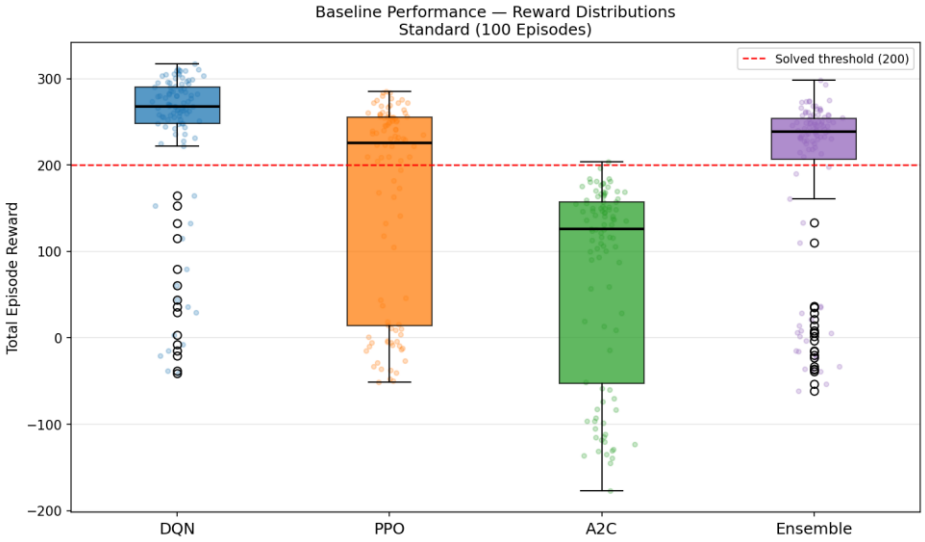

Figure 6: *Histogram of the DQN, PPO, A2C, and Ensemble reward distribution from 100 episodes on the standard LunarLander environment.*

Agent	Mean Reward	Std Dev	Success Rate
DQN	240.62	88.19	85%
Ensemble	191.39	104.18	75%
PPO	160.36	117.85	60%
A2C	71.90	114.10	1%

Table 4: *Baseline agent performances on the standard LunarLander environment*

Table 4 confirms this ranking at evaluation: DQN was the strongest individual agent, with the highest mean reward, lowest standard deviation, highest success rate. The Ensemble agent ranked second,

providing a strong compromise policy. PPO achieved moderate performance, while A2C was clearly weakest, with a mean reward of 71.90 and only 1% success.

Manual inspection of A2C's behaviour showed that the agent had a tendency to drift to the side without correcting to the middle. Even when the agent managed to stay central and land safely, it descended too slowly, resulting in inefficient fuel usage and a cumulative reward below the success threshold. A2C did showcase somewhat competent landing behaviour, despite what the success rate may imply.

Overall, the baseline ordering was clear: DQN > Ensemble > PPO > A2C.

6.2 Frozen-Policy Generalisation

The second stage of the study evaluated how well the trained baseline agents generalised when deployed unchanged in modified environments.

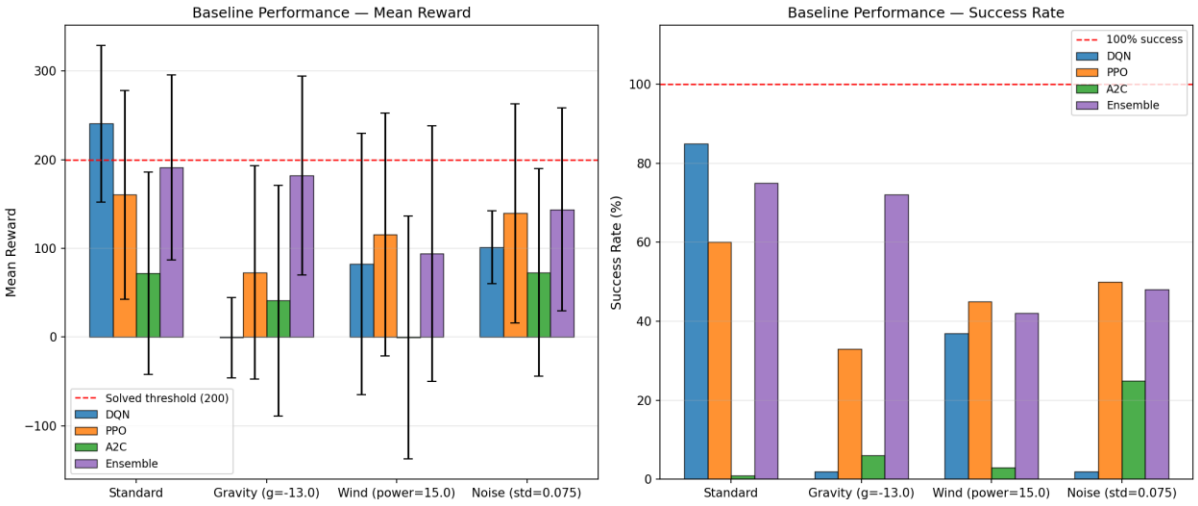

Figure 7: *Bar charts comparing the performances of the agents in the standard and modified environments.*

6.2.1 Gravity

The gravity-modified environment produced one of the strongest shifts in relative agent performance.

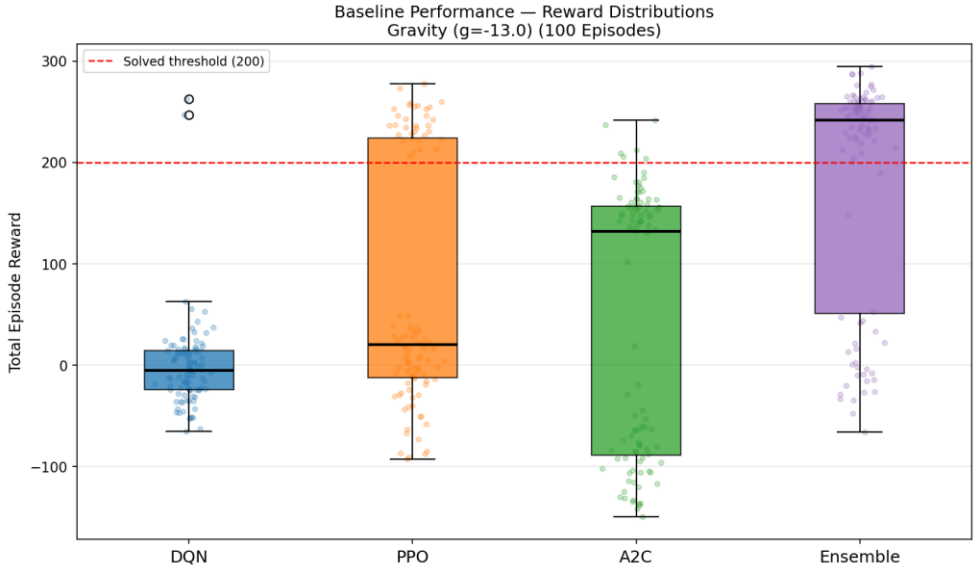

Figure 8: *Histogram of the DQN, PPO, A2C, and Ensemble reward distribution from 100 episodes on the modified gravity LunarLander environment.*

Agent	Mean Reward	Std Dev	Success Rate
Ensemble	182.20	112.05	72%
PPO	72.89	120.26	33%
A2C	40.99	129.90	6%
DQN	-0.72	45.09	2%

Table 5: *Baseline agent performances on the modified gravity LunarLander environment.*

DQN generalised very poorly, collapsing from the strongest baseline agent (85% success) to the weakest under gravity (2% success). PPO became the strongest individual algorithm, while A2C remained weak but generalised fairly well relative to its standard evaluation. The Ensemble agent performed best by a large margin, retaining near-solved performance despite the environmental change.

6.2.2 Wind

The wind environment produced a different ordering.

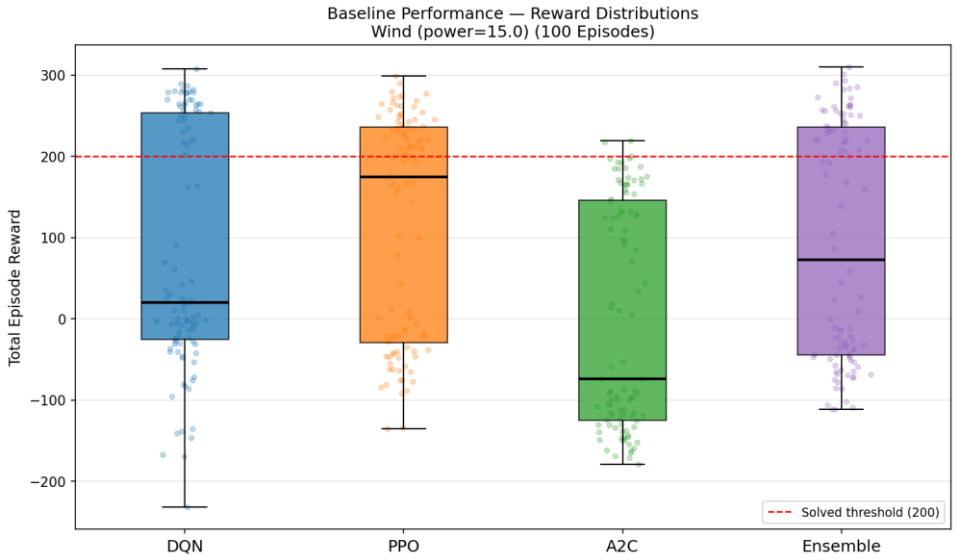

Figure 9: *Histogram of the DQN, PPO, A2C, and Ensemble reward distribution from 100 episodes on the modified wind LunarLander environment.*

Agent	Mean Reward	Std Dev	Success Rate
PPO	115.68	137.04	45%
Ensemble	94.11	144.13	42%
DQN	82.20	147.42	37%
A2C	-0.51	136.73	3%

Table 6: *Baseline agent performances on the modified wind LunarLander environment.*

In this setting, PPO was the strongest agent, followed closely by the Ensemble. While DQN performed moderately, its collapse was much smaller than under gravity. A2C again performed worst, with a near-zero mean reward.

6.2.3 Noise

The noise environment slightly altered the ranking compared to wind.

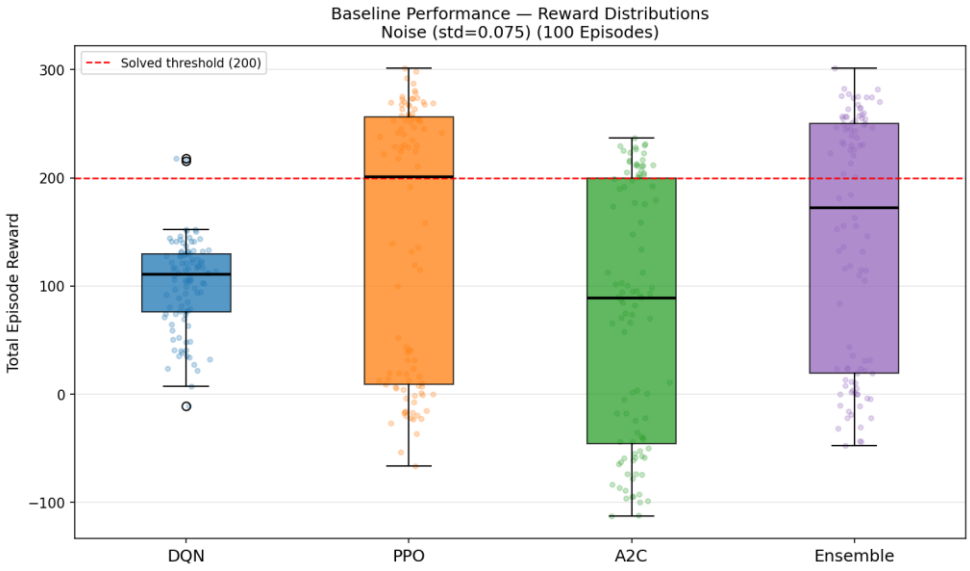

Figure 10: *Histogram of the DQN, PPO, A2C, and Ensemble reward distribution from 100 episodes on the modified noise LunarLander environment.*

Agent	Mean Reward	Std Dev	Success Rate
Ensemble	143.83	144.13	42%
PPO	139.48	123.37	50%
DQN	101.46	40.99	2%
A2C	72.70	117.03	25%

Table 7: *Baseline agent performances on the modified noise LunarLander environment.*

PPO and the Ensemble were again the strongest agents, finishing with very similar performance.

DQN showed moderate average reward but only 2% success, suggesting that it avoided total collapse but was prevented from achieving solved-level landing scores. Manual inspection revealed that DQN would land competently, but would aimlessly waste fuel while on the ground, demonstrating sensitivity to noise.

A2C remained weakest overall, but displayed strong generalisation with it actually outperforming its standard-environment results.

6.3 Transfer Learning

The third stage of this study compared frozen, fine-tuned, and scratch-trained agents in each modified environment to assess whether pretraining improved adaptation speed and final performance.

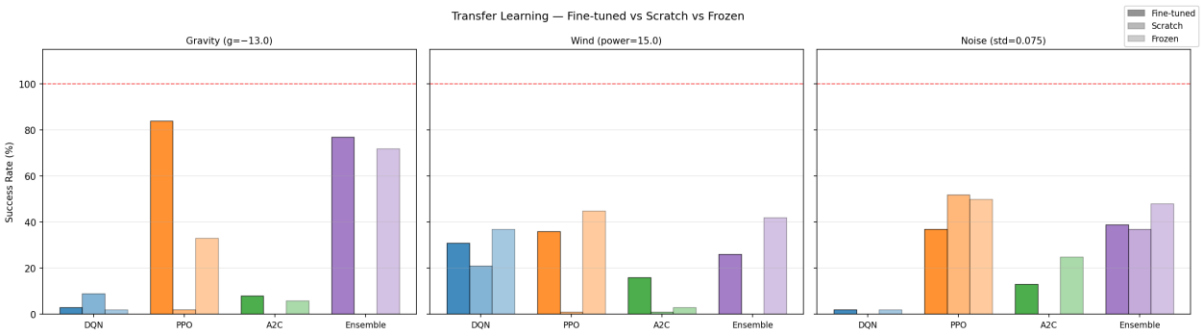

Figure 11: *Bar charts of the success rate of the fine-tuned, scratch-trained, and frozen models for the agents across all environments.*

6.3.1 Gravity

Gravity produced the clearest positive transfer result for PPO, with its fine-tuned model outperforming both the frozen and scratch-trained policies.

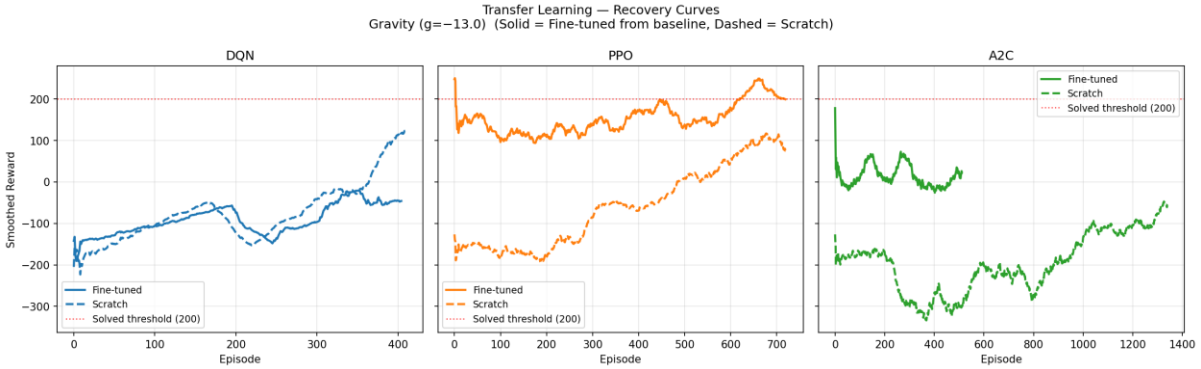

Figure 12: *Learning curves of the fine-tuned and scratch-trained models for DQN, PPO, and A2C in the gravity environment*

Agent	Frozen Mean	Fine-tuned Mean	Scratch Mean	Key Outcome
PPO	72.89	206.77	-30.00	Strong positive transfer
Ensemble	182.20	197.07	-119.64	Weak positive transfer
A2C	40.99	-21.15	-775.88	Frozen performs best
DQN	-0.72	-63.03	41.88	Negative transfer

Table 8: *Frozen, fine-tuned, and scratch-trained performances on the modified gravity LunarLander environment.*

By contrast, DQN showed negative transfer. A2C gained some benefit from transfer relative to scratch training, but did not surpass its frozen policy. The Ensemble improved only slightly with fine-tuning, as its frozen policy was already strong in this environment.

6.3.2 Wind

Transfer effects under wind were weaker and less consistent than under gravity.

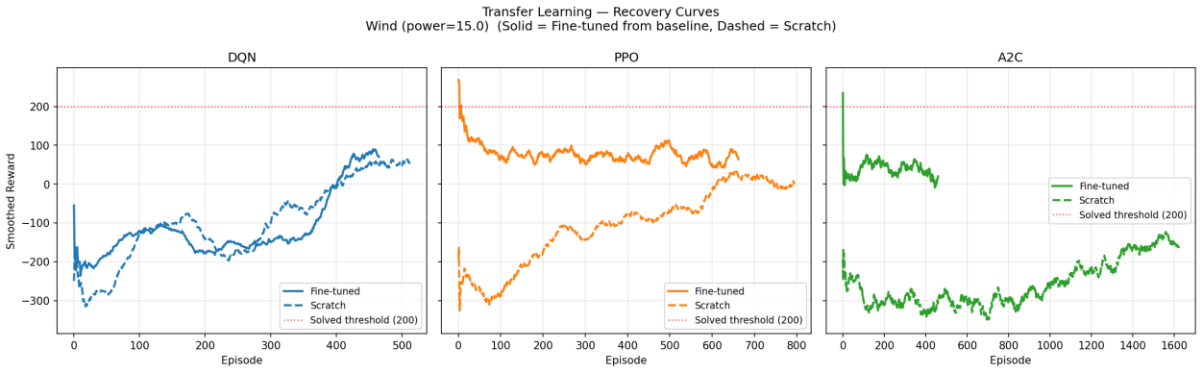

Figure 13: *Learning curves of the fine-tuned and scratch-trained models for DQN, PPO, and A2C in the wind environment*

Agent	Frozen Mean	Fine-tuned Mean	Scratch Mean	Key Outcome
DQN	82.20	95.12	59.40	Weak positive transfer
A2C	-0.51	25.48	-1288.85	Weak positive transfer
PPO	115.68	85.63	-76.15	Frozen performs best
Ensemble	94.11	68.27	-89.67	Frozen performs best

Table 9: *Frozen, fine-tuned, and scratch-trained performances on the modified wind LunarLander environment.*

For DQN and A2C, fine-tuning slightly improved model performance. While the frozen PPO and Ensemble policies showed the best generalisation potential, their fine-tuned models struggled to adapt above this baseline.

6.3.3 Noise

The noise environment provided the weakest evidence for transfer learning.

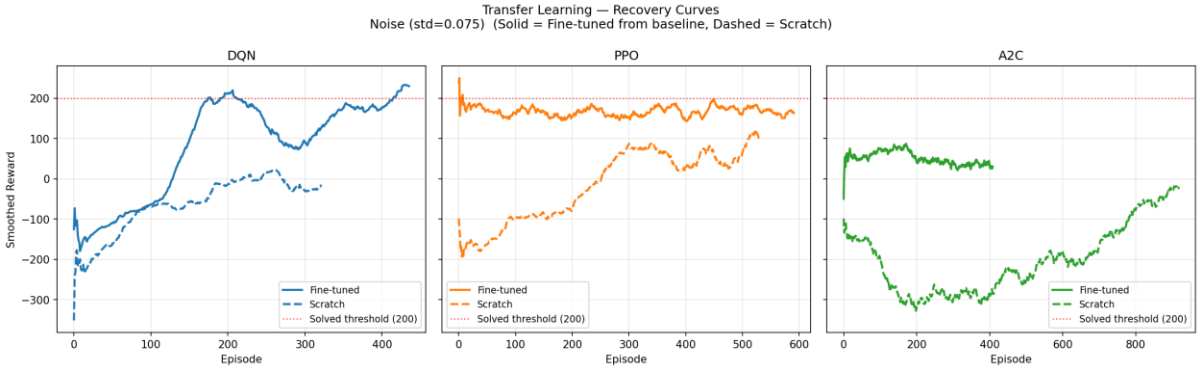

Figure 14: *Learning curves of the fine-tuned and scratch-trained models for DQN, PPO, and A2C in the noise environment*

Agent	Frozen Mean	Fine-tuned Mean	Scratch Mean	Key Outcome
DQN	101.46	113.37	14.28	Weak positive transfer
A2C	72.73	4.47	-550.80	Frozen performs best
Ensemble	143.83	121.96	140.01	Negative transfer
PPO	139.48	116.50	139.35	Negative transfer

Table 10: *Frozen, fine-tuned, and scratch-trained performances on the modified noise LunarLander environment.*

DQN showed only limited benefit from fine-tuning. Results from PPO and Ensemble show that fine-tuning negatively impacts performance. A2C substantially improved over scratch through fine-tuning, but remained below its frozen policy.

7 DISCUSSION

The results answer RS1 by showing that strong baseline performance did not reliably translate into strong generalisation. DQN was the strongest agent in the standard environment, but its performance deteriorated sharply in the modified environments. PPO was the most robust individual algorithm, and the Ensemble showed the strongest robustness overall, suggesting that combining multiple policies may reduce vulnerability to individual failure modes under distribution shift, and increase generalisation. These results indicate that source-task competence and generalisation should be treated as distinct properties.

The results for RS2 were more mixed. Transfer learning was highly dependent on both the algorithm and the type of perturbation. PPO showed clear positive transfer under gravity, but frequently failed to outperform the frozen policy in the wind and noise environments. A2C and DQN's poor results across the modified environments suggest that pretraining provided limited and inconsistent benefit, and that training from scratch may be preferable in some settings. The Ensemble's fine-tuning results were extremely similar to PPO's across the board, perhaps reflecting the limitation of how tiebreaks were implemented in decision-making. Overall, RS2 shows that transfer effectiveness depends strongly on both the algorithm and the nature of the environmental change.

These findings are consistent with the algorithms' design differences. DQN appears to have learned a highly effective but brittle policy closely tied to the training distribution. This likely reflects DQN's off-policy learning process, which relies on Q-value estimates computed from the experience replay from the original environment. PPO's more stable update mechanism via clipped objectives may have restrained the agent from specialising too far to a single environment, supporting both stronger robustness and safe fine-tuning. A2C remained the weakest method overall, but did benefit from pretraining relative to scratch training. This may have resulted from A2C's low learning efficiency as opposed to its generalisation and transfer abilities.

7.1 LIMITATIONS

This study is limited by its use of a single benchmark (LunarLander), a small algorithm set, custom hyperparameter tuning, and unequal training budgets. The perturbation strengths were also not varied systematically, and no formal significance testing was conducted. Finally, each experiment was conducted with a single random seed, so some observed differences may reflect stochastic training variance rather than concrete algorithmic differences. The findings should therefore be interpreted as evidence from a focused comparative study rather than as a definitive ranking of RL methods.

8 CONCLUSION

This project compared DQN, PPO, A2C, and an ensemble agent on the LunarLander task to investigate generalisation under environmental change, and the value of transfer learning. The results for *RS1* showed that a strong baseline performance did not reliably predict robustness: although DQN was the strongest agent in the standard environment, PPO and the Ensemble were generally more robust in the modified conditions. For *RS2*, transfer learning was found to be highly dependent on both algorithm and perturbation type, with PPO showing strong positive transfer under gravity, while other cases showed weak or negative transfer.

Overall, the findings show that reinforcement learning agents should be evaluated not only by source-task performance, but also by how well they remain effective under distribution shift, and how successfully they adapt when fine-tuned. In this study, PPO was the most reliable individual agent across changed environments, while the Ensemble provided the strongest robustness overall.

8.1 FUTURE WORK

Future work could improve the rigour, scope, and adaptability of the study. All experiments should be repeated with confidence intervals and statistical tests to determine whether the observed differences are significant and robust. The perturbations should be varied systematically across several strengths to analyse how performance degrades under increasing distribution shift. Such data collection was beyond the scope of this project. Additionally, the transfer learning study could also be extended through domain randomisation or more advanced adaptation methods. The ensemble could be improved using confidence-weighted or learned voting rather than a fixed majority rule.

9 BIBLIOGRAPHY

[1] 'Gymnasium Documentation'. Accessed: May 10, 2026. [Online]. Available: https://gymnasium.farama.org/environments/box2d/lunar_lander.html

[2] 'Gymnasium Documentation'. Accessed: May 10, 2026. [Online]. Available: https://gymnasium.farama.org/index.html

[3] D. Shen, 'Comparison of Three Deep Reinforcement Learning Algorithms for Solving the Lunar Lander Problem', in *Proceedings of the 2023 International Conference on Data Science, Advanced Algorithm and Intelligent Computing (DAI 2023)*, vol. 180, B. H. Ahmad, Ed., in Advances in Intelligent Systems Research, vol. 180. , Dordrecht: Atlantis Press International BV, 2024, pp. 187–199. doi: 10.2991/978-94-6463-370-2_21.

[4] A. Awasthi, 'Evaluating Reinforcement Learning algorithms forLunarLander-v2: A Comparative Analysis', Apr. 15, 2025, *In Review*. doi: 10.21203/rs.3.rs-5939959/v2.

[5] J. M. P. D. Kavlakoglu Eda, 'What is reinforcement learning? | IBM'. Accessed: May 10, 2026. [Online]. Available: https://www.ibm.com/think/topics/reinforcement-learning

[6] '11 Markov Decision Processes – 6.390 - Intro to Machine Learning'. Accessed: May 10, 2026. [Online]. Available: https://introml.mit.edu/notes/mdp.html

[7] T. Fujimoto, J. Suetterlein, S. Chatterjee, and A. Ganguly, 'Assessing the Impact of Distribution Shift on Reinforcement Learning Performance', 2024, *arXiv*. doi: 10.48550/ARXIV.2402.03590.

[8] V. Mnih *et al.*, 'Playing Atari with Deep Reinforcement Learning', 2013, *arXiv*. doi: 10.48550/ARXIV.1312.5602.

[9] 'Deep Q-Network (DQN) Agent - MATLAB & Simulink'. Accessed: May 11, 2026. [Online]. Available: https://www.mathworks.com/help/reinforcement-learning/ug/dqn-agents.html

[10] N. a V. for Y. G. A. C. is a managed vector database built on M. perfect for building G. applications T. Free, 'What is a Q-function in RL?' Accessed: May 13, 2026. [Online]. Available: https://milvus.io/ai-quick-reference/what-is-a-qfunction-in-rl

[11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, ‘Proximal Policy Optimization Algorithms’, 2017, *arXiv*. doi: 10.48550/ARXIV.1707.06347.
[12] ‘Proximal Policy Optimization | huggingface/deep-rl-class’, DeepWiki. Accessed: May 11, 2026. [Online]. Available: https://deepwiki.com/huggingface/deep-rl-class/9-proximal-policy-optimization
[13] V. Mnih *et al.*, ‘Asynchronous Methods for Deep Reinforcement Learning’, 2016, doi: 10.48550/ARXIV.1602.01783.
[14] ‘Advantage Actor Critic (A2C)’. Accessed: May 11, 2026. [Online]. Available: https://huggingface.co/blog/deep-rl-a2c
[15] A. KOHLI, *ankurkohli007/A2C-PPO-Reinforcement-Learning_Machine-Learning-for-Robotics-II*. (Jul. 03, 2025). Jupyter Notebook. Accessed: May 11, 2026. [Online]. Available: https://github.com/ankurkohli007/A2C-PPO-Reinforcement-Learning_Machine-Learning-for-Robotics-II
[16] Y. Song *et al.*, ‘Ensemble reinforcement learning: A survey’, *Appl. Soft Comput.*, vol. 149, p. 110975, Dec. 2023, doi: 10.1016/j.asoc.2023.110975.
[17] J. M. P. D. Kavlakoglu Eda, ‘What is transfer learning? | IBM’. Accessed: May 11, 2026. [Online]. Available: https://www.ibm.com/think/topics/transfer-learning
[18] H. Ahn, J. Hyeon, Y. Oh, B. Hwang, and T. Moon, ‘Reset & Distill: A Recipe for Overcoming Negative Transfer in Continual Reinforcement Learning’, 2024, *arXiv*. doi: 10.48550/ARXIV.2403.05066.
[19] J. Teoh, P. Varakantham, and P. Vamplew, ‘On Generalization Across Environments In Multi-Objective Reinforcement Learning’, 2025, *arXiv*. doi: 10.48550/ARXIV.2503.00799.
[20] C. Benjamins *et al.*, ‘Contextualize Me -- The Case for Context in Reinforcement Learning’, Jun. 02, 2023, *arXiv*: arXiv:2202.04500. doi: 10.48550/arXiv.2202.04500.
[21] Z. Zhu, K. Lin, A. K. Jain, and J. Zhou, ‘Transfer Learning in Deep Reinforcement Learning: A Survey’, 2020, *arXiv*. doi: 10.48550/ARXIV.2009.07888.
[22] Antonin RAFFIN *et al.*, *DLR-RM/stable-baselines3: v2.8.0: Dropped Python 3.9, added Python 3.13 support, MaskablePPO bug fix, default hyperparams for unlisted env in the RL Zoo, Markdown doc*. (Apr. 01, 2026). Zenodo. doi: 10.5281/ZENODO.8123988.
[23] *numpy: Fundamental package for array computing in Python*. C, Python. Accessed: May 12, 2026. [MacOS, Microsoft :: Windows, POSIX, Unix]. Available: https://numpy.org
[24] *matplotlib: Python plotting package*. Python. Accessed: May 12, 2026. [Online]. Available: https://matplotlib.org
[25] Z. Xue *et al.*, ‘State Regularized Policy Optimization on Data with Dynamics Shift’, 2023, *arXiv*. doi: 10.48550/ARXIV.2306.03552.
[26] A. Mahajan and A. Zhang, ‘Generalization Across Observation Shifts in Reinforcement Learning’, 2023, *arXiv*. doi: 10.48550/ARXIV.2306.04595.
[27] V. Mnih *et al.*, ‘Human-level control through deep reinforcement learning’, *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015, doi: 10.1038/nature14236.